CargoLink

Marketplace de covoiturage de fret entre PME et transporteurs

CONNECTER • OPTIMISER • AVANCER ENSEMBLE

ESPRIT — Classe 5TWIN — Projet Django

Dossier de validation

Correspondance avec la grille d'évaluation et répartition du travail de l'équipe

Équipe	Rôle principal
Ahmed Saidani	Intelligence artificielle (les 5 IA et leur infrastructure), comptes & contrôle d'accès, modération, intégration & déploiement
Fedi Ben Khalifa	Annonces de fret, trajets & véhicules, recherche & filtres
Farah Maiti	Réservations & machine à états, messagerie
Wissem Ferjani	Avis & réputation, notifications, tableaux de bord, accessibilité

Élément	Valeur
Dépôt Git	github.com/ahmedsaidani1/django-python-5twin2
Stack	Python 3.12 · Django 5.2 · PostgreSQL 16 · Redis 7 + Celery 5 · Bootstrap 5 · scikit-learn
Déploiement	Docker / docker compose (local) · Render, offre gratuite (Blueprint render.yaml)
Date	27/10/2026

1. Synthèse

CargoLink met en relation des **PME** qui ont des marchandises à expédier et des **transporteurs indépendants** qui ont de la capacité libre sur leurs trajets (problème des trajets à vide). La PME publie une annonce de fret, le transporteur publie ses trajets ; chacun peut envoyer une demande à l'autre, qui l'accepte ou la refuse, puis la livraison est suivie jusqu'au statut « livrée » et la PME note le transporteur. Un administrateur modère les contenus. Cinq fonctionnalités d'intelligence artificielle, mesurées par des métriques, assistent chaque étape.

Synthèse par module de la grille

Module	Barème	Ce qui est livré	Statut
1. Fonctionnalités & CRUD	2	CRUD complet (annonces, trajets, véhicules, profils) ; 11 fonctionnalités du cahier des charges ; validations serveur + HTML5	**Réalisé**
2. Tests & BNF	4 *	155 tests (pytest et manage.py test), couverture 98,4 % ; sécurité OWASP ; matrice de permissions sur toutes les vues ; pages 403/404/500	**Réalisé**
3. Accessibilité	2	Sémantique, un seul h1 par page, ARIA, contrastes calculés ≥ 4,5:1, navigation clavier complète. Audit Lighthouse/axe : procédure prête, rapport à joindre	**À finaliser**
4. Intelligence artificielle	4	5 IA intégrées, asynchrones (Celery/Redis), latence mesurée, métriques chiffrées et commentées pour chacune	**Réalisé**
5. UX / UI	1	Charte CargoLink, responsive, navigation par rôle, cartes, badges de statut, états vides, toasts	**Réalisé**
6. Questions / QCM	8	Fiche « à maîtriser » par membre (docs/OWNERSHIP.md) et réponses aux questions probables (docs/JURY_QA.md)	Préparation individuelle
Git (condition)	—	Dépôt GitHub, 61 commits conventionnels, une branche par périmètre fusionnée en --no-ff	**Réalisé**
Intégration & déploiement (condition)	—	Dockerfile + docker compose (web, db, redis, worker) testés ; Blueprint Render synchronisé ; CI GitHub Actions configurée (première exécution à vérifier)	**À finaliser**

* Le tableau récapitulatif de la grille indique 3 points pour « Tests & BNF » mais le détail de la section en totalise 4 (2 + 2) ; ce dossier suit le détail.

Fonctionnalités principales

#	Fonctionnalité	Rôle(s)
1	Inscription par rôle, connexion/déconnexion, mot de passe oublié, profil entreprise ou transporteur	Tous
2	Publication d'annonces de fret et de trajets (prix au kg ou forfait), gestion des véhicules	PME / Transporteur
3	Recherche : ville de départ/arrivée, date, volume, type de marchandise, texte libre, pagination	Tous
4	Mise en relation : demande depuis un trajet ou offre depuis une annonce, avec message et prix proposé	PME / Transporteur
5	Machine à états en_attente → acceptée → en cours → livrée (ou refusée / annulée), historique	PME / Transp.
6	Tableau de bord par rôle : annonces, demandes reçues/envoyées, livraisons, notifications, statistiques	Tous
7	Messagerie : un fil de discussion par réservation	PME / Transporteur
8	Avis après livraison, note moyenne et score de réputation du transporteur	PME
9	Notifications in-app : nouvelle demande, changement de statut, nouveau message	Tous
10	Signalement d'une annonce, d'un trajet ou d'un utilisateur ; modération ; admin Django complet	Admin
11	Page d'accueil : problème, solution, valeur pour chaque acteur, appels à l'action	Public

2. Correspondance détaillée avec la grille

Module 1 — Fonctionnalités & CRUD (2 pts)

Sous-critère (pts)	Réalisation dans CargoLink	Preuve à montrer
Opérations de base (1)	Create / Read / Update / Delete sur les annonces de fret, les trajets et les véhicules ; mise à jour des profils ; les réservations suivent un cycle de vie (création, transitions, annulation) au lieu d'être supprimées, pour garder l'historique.	Démo : publier, modifier, supprimer une annonce et un trajet
Cohérence fonctionnelle (1)	Les 11 fonctionnalités du cahier des charges sont présentes. Validations serveur (date limite après l'enlèvement, poids et volume positifs, capacité du véhicule et du trajet, dates non passées) et client (attributs HTML5 required, min, type=date, pattern). Toutes les pages ont été parcourues automatiquement pour chaque rôle sans erreur.	Scénario de démo du README ; tests apps/*/tests/

Module 2 — Tests & besoins non fonctionnels (4 pts)

Sous-critère (pts)	Réalisation dans CargoLink	Preuve à montrer
Tests unitaires (0,75)	Modèles et validations (clean(), contraintes), les 36 couples de statuts de la machine à états (valides/invalides, bon/mauvais acteur), capacité, prix, matching, sentiment, fraude, extraction d'annonce.	apps/bookings/tests/test_status_machine.py, apps/*/tests/
Tests d'intégration / API (0,75)	Client Django : codes HTTP, contexte, redirections, réponses JSON des endpoints IA (client LLM simulé avec unittest.mock), commandes de gestion.	apps/core/tests/test_ai_infra.py
Couverture & exécution (0,5)	155 tests verts avec **pytest** et avec **python manage.py test** ; couverture **98,4 %** (seuil bloquant 80 %) ; rapport HTML ; exécutés aussi sur PostgreSQL.	pytest puis htmlcov/index.html
Sécurité OWASP (0,75)	CSRF actif (formulaires et appels fetch), ORM uniquement, mots de passe hachés (PBKDF2), échappement XSS testé, verrouillage après 5 échecs de connexion (django-axes, HTTP 429), DEBUG=False, HTTPS, HSTS et cookies Secure en production, secrets dans .env (modèle .env.example).	config/settings/prod.py ; manage.py check --deploy
Rôles & permissions (0,75)	Groupes PME / Transporteur / Administrateur et leurs permissions créés par migration de données ; RoleRequiredMixin et OwnerRequiredMixin sur chaque vue. Un test parcourt toutes les vues : anonyme → connexion, mauvais rôle → 403, non-propriétaire → 403, accès direct par URL impossible.	apps/core/tests/test_permissions.py
Robustesse & erreurs (0,5)	Pages 403 / 404 / 500 personnalisées, try/except autour des appels externes (LLM, broker, notifications) avec journalisation, validation de toutes les entrées.	templates/errors/ ; apps/core/tasks.py

Module 3 — Accessibilité web (2 pts)

Sous-critère (pts)	Réalisation dans CargoLink	Preuve à montrer
Structure sémantique (0,5)	header, nav, main, footer ; exactement un h1 par page (vérifié automatiquement) ; hiérarchie h1 → h3 cohérente ; tableaux avec caption et scope.	templates/base.html
ARIA & textes alternatifs (0,5)	alt sur les images, icônes aria-hidden, boutons icône avec aria-label, erreurs liées par aria-describedby et aria-invalid, aria-current dans la navigation, régions aria-live pour les messages et les notifications, aria-busy pendant les appels IA.	templates/partials/form_field.html
Contraste & lisibilité (0,5)	Ratios calculés : texte 14,3:1, texte secondaire 6,5:1, blanc sur marine 14,0:1, badges de statut ≥ 6,4:1. Le jaune n'est jamais utilisé comme texte sur fond blanc.	docs/accessibility.md

Sous-critère (pts)	Réalisation dans CargoLink	Preuve à montrer
Navigation clavier (0,5)	Lien d'évitement, focus visible (:focus-visible), modales de confirmation qui piègent le focus et se ferment avec Échap. **À finaliser :** exécuter et joindre l'audit Lighthouse / axe (procédure prête).	Démo au clavier ; rapport Lighthouse

Module 4 — Intelligence artificielle (4 pts)

Sous-critère (pts)	Réalisation dans CargoLink	Preuve à montrer
Pertinence (0,75)	Chaque IA répond à un besoin réel : remplir une annonce depuis une phrase, proposer un prix juste, trouver le bon trajet, mesurer la réputation, protéger la plateforme du spam.	Section 3
Intégration technique (0,75)	Vue dédiée POST /ai/<type>/ ; client LLM indépendant du fournisseur (timeout, retries, exceptions propres) ; clé et fournisseur dans les variables d'environnement ; modèles locaux si aucune clé n'est fournie.	apps/core/ai/llm_client.py
Expérience utilisateur (0,5)	Spinner, aria-busy, statut annoncé aux lecteurs d'écran, message d'erreur explicite en français en cas d'échec ou de délai.	static/js/app.js
Métriques (1)	Précision par champ, MAE / RMSE / R^2, Precision@5 / NDCG@5, précision / rappel / F1 avec matrices de confusion ; comparaison à des références ; limites expliquées.	docs/ai/*.md ; evaluate_*
Performance & asynchronisme (1)	Chaque appel IA est une tâche Celery (Redis) enregistrée dans AIJob avec sa latence ; la requête HTTP répond immédiatement (202) et le navigateur interroge /ai/jobs/<id>/. Latence moyenne et p95 dans chaque rapport.	docker compose (worker Celery)

Module 5 — UX / UI (1 pt)

Sous-critère (pts)	Réalisation dans CargoLink	Preuve à montrer
Cohérence & ergonomie (1)	Charte CargoLink (marine #0B2A5B, jaune #FFB400), interface responsive (Bootstrap 5), barre de navigation propre à chaque rôle, cartes, badges de statut, états vides avec action, notifications toast, fil d'Ariane.	Démo sur mobile et ordinateur

Conditions éliminatoires

Condition	Situation	Statut
Git (sinon note individuelle / 2)	Dépôt GitHub ; 61 commits conventionnels (feat, fix, test, docs, build, ci) ; une branche par périmètre fusionnée dans main avec --no-ff. Chaque membre poursuit ses commits sur sa branche avec sa propre identité Git.	**Réalisé**
Intégration & déploiement (sinon note d'équipe = 0)	Dockerfile et docker-compose.yml (gunicorn, PostgreSQL, Redis, worker Celery) testés de bout en bout ; WhiteNoise ; Blueprint Render (offre gratuite) synchronisé (base + service web). À vérifier : première exécution de la CI GitHub Actions et URL publique en ligne.	**À finaliser**

3. Intelligence artificielle : modèles et résultats

Les cinq IA fonctionnent sans connexion : elles reposent sur scikit-learn et sur des règles explicables. Un fournisseur LLM (API compatible OpenAI ou Anthropic) peut être activé par variable d'environnement pour l'extraction d'annonces ; en cas d'absence de clé ou d'échec, le modèle local prend le relais. Chaque commande **evaluate_<ia>** calcule les métriques et la latence (moyenne et p95) et écrit un rapport dans docs/ai/.

IA	Modèle	Jeu d'évaluation	Résultats	Responsable
1. Annonce intelligente	Extracteur de règles en français (ou LLM si clé)	50 textes annotés à la main	Précision par champ **98,6 %**	Ahmed Saidani
2. Suggestion de prix	GradientBoostingRegressor (400 arbres)	800 trajets de test (données synthétiques)	MAE ≈ **39 TND**, RMSE ≈ 74, R^2 **0,968**	Ahmed Saidani
3. Matching	Score pondéré : trajet, date, capacité, prix, réputation	40 annonces × 15 trajets annotés	P@5 **0,445** (max 0,48), NDCG@5 **0,917**	Ahmed Saidani
4. Sentiment des avis	TF-IDF (mots + caractères) + régression logistique	75 avis rédigés à la main	F1 macro **0,867**, matrice de confusion	Ahmed Saidani
5. Fraude / spam	RandomForest sur 13 variables explicables	480 annonces (gabarits disjoints)	Précision **1,00**, rappel **0,83**, F1 **0,91**	Ahmed Saidani

Analyse et limites (présentées honnêtement au jury)

- **Données synthétiques** pour le prix, le matching et la fraude, faute d'historique réel : chaque rapport le précise. Les jeux de test des annonces et des avis sont rédigés à la main et n'ont jamais servi à l'entraînement.
- **Sentiment** : le vocabulaire d'entraînement a été élargi une fois après une première évaluation (F1 0,774 → 0,867) ; le score est donc probablement un peu optimiste.
- **Fraude** : au seuil 0,5, aucune fausse alerte ; baisser le seuil augmente le rappel (0,93 à 0,3). L'IA signale, l'administrateur décide : aucune suppression automatique.
- **Matching** : la Precision@5 est plafonnée par le nombre de trajets pertinents par annonce (borne 0,48) ; le score est affiché à l'utilisateur avec le détail de chaque critère.
- **Asynchronisme** : en local et avec docker compose, Celery + Redis ; sur l'offre gratuite de Render (sans worker), la tâche s'exécute dans un thread d'arrière-plan : la page ne bloque jamais.

4. Répartition du travail par membre

Chaque membre est responsable d'un périmètre complet : modèles, vues, formulaires, templates et les tests associés. L'ensemble de l'intelligence artificielle est porté par Ahmed Saidani. Il présente cette partie à la soutenance et répond aux questions du jury sur ce code. Le socle commun (configuration, mixins d'accès, gabarit de base, styles) est partagé et relu en équipe.

Ahmed Saidani — Intelligence artificielle, comptes, modération, intégration & déploiement

Branche Git : **member1/accounts-moderation-fraud-deploy et parties IA des autres branches**

Domaine	Contenu
IA — infrastructure	Client LLM multi-fournisseur (timeout, retries, repli local), modèle AIJob, tâches Celery + Redis, endpoints /ai/ avec réponse 202 et polling, repli en thread sans broker, entraînement des modèles (train_ai_models), outils d'évaluation (latence moyenne / p95, rapports, matrices de confusion)
IA — 5 fonctionnalités	Annonce intelligente, suggestion de prix, matching annonces ↔ trajets, sentiment des avis et score de réputation, détection de fraude / spam ; jeux de données data/ai/, commandes evaluate_*, rapports docs/ai/
Comptes & accès	Modèle User avec rôle (créé avant la première migration), profils PME et transporteur, groupes et permissions par migration de données, RoleRequiredMixin, OwnerRequiredMixin, django-axes, matrice de tests de permissions
Modération	Signalements (annonce, trajet, utilisateur), file de modération, masquage et désactivation, admin Django configuré pour tous les modèles
Intégration & déploiement	Dockerfile, docker-compose (web, PostgreSQL, Redis, worker), settings de production, Blueprint Render, CI GitHub Actions, commande seed_demo
IA	Les 5 IA : précision par champ (98,6 %) ; régression GradientBoosting (MAE ≈ 39 TND, R^2 0,968) ; matching (P@5 0,445, NDCG@5 0,917) ; sentiment TF-IDF + régression logistique (F1 0,867) ; fraude RandomForest (F1 0,91)
Partie de la démonstration	Toutes les démonstrations IA (« Remplir avec l'IA », « Suggérer un budget », matching, réputation, annonce signalée), commandes evaluate_*, worker Celery ; inscription et rôles, accès par URL directe (403), modération, admin Django, docker compose, CI et déploiement Render
Notions à maîtriser	Celery + Redis, réponse 202 et polling, gestion des erreurs d'API ; Gradient Boosting, régression logistique, TF-IDF, forêt aléatoire ; MAE / RMSE / R^2, précision / rappel / F1, matrice de confusion, Precision@5 / NDCG, choix d'un seuil, biais d'évaluation ; AbstractUser, mixins d'accès, CSRF, settings de production, Docker, Render

Fedi Ben Khalifa — Annonces de fret, trajets & véhicules, recherche

Branche Git : **member2/shipments-smart-annonce et member3/trips-price**

Domaine	Contenu
Annonces de fret	Modèle FreightRequest et ses validations (date limite, poids, volume, villes), CRUD complet, page détail, « mes annonces »
Trajets & véhicules	Modèles Vehicle et Trip (capacité, tarification au kg ou forfait), CRUD, contrôle de la capacité du véhicule et du poids déjà réservé
Recherche	Filtres départ / arrivée / date / volume / type / texte, pagination, index en base
Référentiel	Villes tunisiennes et calcul de distance (geo.py)
IA	Intègre dans les formulaires d'annonce et de trajet les boutons IA fournis par Ahmed Saidani
Partie de la démonstration	CRUD d'une annonce et d'un trajet avec erreurs de validation, gestion des véhicules, recherche filtrée et pagination
Notions à maîtriser	Modèles et migrations, ModelForms et clean(), validation HTML5 + serveur, vues génériques (Create / Update / Delete / List / Detail), select_related et index, pagination

Farah Maiti — Réservations, machine à états, messagerie

Branche Git : **member4/bookings-matching-messaging**

Domaine	Contenu
Réservations	Demande depuis un trajet (PME) ou offre depuis une annonce (transporteur), prix proposé, message, listes reçues / envoyées et filtres par statut
Machine à états	Transitions contrôlées par acteur, transactions atomiques, réservation et libération de capacité, refus automatique des offres concurrentes, historique des statuts
Messagerie	Fil de discussion par réservation, messages non lus, boîte de réception
Interface	Page détail d'une réservation, modales de confirmation (refus / annulation)
IA	Affiche dans la page d'annonce et de trajet les résultats du matching IA fourni par Ahmed Saidani
Partie de la démonstration	Envoi d'une demande, acceptation, démarrage et livraison, historique des statuts, transition interdite refusée, messagerie
Notions à maîtriser	Machine à états (table de transitions), transaction.atomic et select_for_update, contrôle d'accès objet, formulaires avec règles métier, annotations ORM (Count, Max)

Wissem Ferjani — Avis, notifications, tableaux de bord, accessibilité

Branche Git : **member5/reviews-notifs-dashboard-sentiment**

Domaine	Contenu
Avis	Avis après livraison (note et commentaire), contrôle « une seule note par livraison », liste des avis
Notifications	Notifications in-app (demande, statut, message, avis, modération), compteur dans la barre de navigation, « tout marquer comme lu », région aria-live
Tableaux de bord	Tableaux PME, transporteur et administrateur avec statistiques
Accueil & accessibilité	Page d'accueil, pages d'erreur 403 / 404 / 500, checklist WCAG AA, contrastes, navigation clavier, procédure Lighthouse / axe
IA	Affiche la note moyenne et le score de réputation calculés par l'IA de sentiment d'Ahmed Saidani
Partie de la démonstration	Page d'accueil, dépôt d'un avis, notifications, tableaux de bord, démonstration au clavier et audit d'accessibilité
Notions à maîtriser	Context processor, agrégations ORM (Avg, Sum, Count) ; WCAG AA : repères sémantiques, un seul h1, aria-describedby, aria-live, contrastes, focus visible, modales

5. Annexes

Lancer le projet

Objectif	Commande
Installation (venv)	python -m venv .venv · .venv\Scripts\activate · pip install -r requirements-dev.txt
Base, modèles IA, démo	python manage.py migrate · python manage.py train_ai_models · python manage.py seed_demo
Serveur de développement	python manage.py runserver → http://127.0.0.1:8000
Pile complète (Celery, Redis, PostgreSQL)	docker compose up --build → http://localhost:8000
Tests et couverture	pytest (rapport htmlcov/index.html) · python manage.py test
Évaluation des IA	python manage.py evaluate_smart_annonce | evaluate_price | evaluate_matching | evaluate_sentiment | evaluate_fraud

Comptes de démonstration

Rôle	Identifiant	Mot de passe
Administrateur	admin	Admin2026!
PME	pme_sfax · pme_monastir · pme_bizerte	Demo2026!
Transporteur	trans_sahel · frigo_sud · capbon_log	Demo2026!

Ces mots de passe concernent uniquement les données de démonstration ; ils doivent être changés sur toute instance publique.

Documentation du dépôt

Document	Contenu
README.md	Installation, architecture (diagramme), tests, IA, déploiement, scénario de démonstration
docs/OWNERSHIP.md	Répartition détaillée des fichiers par membre et règles de contribution
docs/JURY_QA.md	Questions probables du jury et réponses argumentées
docs/accessibility.md	Checklist WCAG AA, contrastes, procédure Lighthouse et axe
docs/ai/	Rapports d'évaluation des cinq IA et matrices de confusion